Si, i due livelli di dichiarazione proposti al punto 1) risolvono il punto.

La dichiarazione standard al punto 2) corrisponde quindi effettivamente la file di linguaggio runtime.yaml (presumibilmente con sintassi analoga anche per il runtime.yaml di progetto) ? se si, questa sintassi e' da considerarsi specifica per il caso d'uso in oggetto e richiede un parser dedicato da implementare in Perl ?

Sì, esattamente: la dichiarazione del punto 2 è il contenuto proposto per `langs/Python/runtime.yaml`. Il file `.docker-dev/runtime.yaml` del progetto usa lo stesso schema, ma dichiara normalmente soltanto aggiunte o personalizzazioni consentite.

La sintassi è specifica di `docker-dev`, ma questo non richiede un **parser YAML dedicato**: richiede un interprete dello schema, da implementare in Perl sopra il parser YAML esistente.

1. YAML standard, schema applicativo specifico

Occorre distinguere due livelli:

Livello	Esempio	Responsabilità
Sintassi YAML	Mapping, liste, stringhe, booleani	`YAML::XS`
Schema `docker-dev`	`mounts`, `volumes`, `protected`, `preparation`	Codice Perl di `docker-dev`

Per esempio:

```
schema_version: 1

mounts:
  - id: datasets
    type: bind
    source: ./datasets
    target: /datasets
    read_only: true
```

Il parser YAML converte il documento in strutture Perl. Il codice applicativo verifica poi che:

- `mounts` sia una lista;
- ogni mount abbia un identificatore;
- `type` sia un tipo supportato;
- `source` e `target` siano validi;

- `read_only` abbia il tipo previsto;
- non ci siano conflitti con altri mount.

Non stiamo quindi creando un nuovo linguaggio di serializzazione: stiamo definendo un formato di configurazione di `docker-dev` espresso in YAML.

2. Stesso schema, regole diverse per i due livelli

I due file possono condividere la stessa struttura:

```
langs/Python/runtime.yaml
<PROJECT_DIR>/.docker-dev/runtime.yaml
```

Ma "stesso schema" non significa che entrambi possano modificare liberamente ogni proprietà.

Per esempio:

Elemento	File di linguaggio	File di progetto
Dichiarare mount	Sì	Sì
Dichiarare volumi	Sì	Sì
Definire mount standard protetti	Sì	Non può rimuovere la protezione di quelli ereditati
Aggiungere ambiente e porte	Sì	Sì
Sostituire `/app/.venv`	Definisce il mount standard	Non implicitamente
Selezionare hook di linguaggio	Sì, secondo il contratto	Non può appropriarsi implicitamente di un hook di linguaggio

Il resolver deve conoscere la provenienza di ciascun contributo e applicare le regole di composizione.

3. I segnaposto richiedono una risoluzione applicativa

Questa espressione proposta:

```
source: "${context.project_dir}"
```

è una normale stringa per il parser YAML. `YAML::XS` non la espande.

Sarà il resolver di `docker-dev` a interpretarla utilizzando il contesto già determinato dal main:

```
"${context.project_dir}"
            ↓
"/home/utente/REP/DEV/Python/foo"
```

Per la prima versione adotterei un insieme limitato di riferimenti:

```
${context.project_dir}
${context.host_user}
${context.volume_name}
${context.project_id}
```

Con queste regole:

- niente espressioni arbitrarie;
- niente esecuzione di comandi;
- errore per riferimenti sconosciuti;
- espansione soltanto nei campi previsti;
- nessuna espansione automatica di qualsiasi variabile dell'ambiente host.

È una funzione di risoluzione delle stringhe, non un secondo parser YAML.

4. Componenti Perl necessari

Il percorso può essere organizzato così:

```
runtime.yaml
     ↓
Loader YAML
     ↓
Validatore dello schema
     ↓
Resolver dei riferimenti e dei percorsi
     ↓
Compositore linguaggio + progetto
     ↓
Piano runtime normalizzato
     ├─ preparazione delle risorse
     ├─ backend legacy
     └─ backend Compose
```

Una possibile organizzazione dei moduli:

```
lib/DockerDev/Runtime/
├── Loader.pm
├── Validator.pm
├── Resolver.pm
├── Composer.pm
└── Plan.pm
```

Non è necessario partire con cinque moduli separati: sono responsabilità logiche che possono inizialmente vivere in meno file.

Pseudocodice

```
my $language_config = load_yaml($language_runtime_path);
my $project_config  = load_optional_yaml($project_runtime_path);

validate_runtime($language_config, origin => 'language');
validate_runtime($project_config,  origin => 'project');

my $resolved_language = resolve_runtime($language_config, $context);
my $resolved_project  = resolve_runtime($project_config,  $context);

my $plan = compose_runtime(
    language => $resolved_language,
    project  => $resolved_project,
);

validate_plan($plan);
```

Il risultato è una struttura Perl comune ai due backend, non una stringa di argomenti Docker e non ancora un documento Compose.

5. Rapporto con Template::Toolkit

Per i file `runtime.yaml` non serve necessariamente Template::Toolkit.

La separazione che suggerisco è:

File	Elaborazione
`langs/Python/runtime.yaml`	Parsing YAML, validazione e risoluzione controllata
`.docker-dev/runtime.yaml`	Parsing YAML, validazione e risoluzione controllata
`compose.yaml.tt`	Rendering Template::Toolkit dal piano risolto
`compose.yaml` generato	Parsing/validazione del risultato e verifica Compose

Questo evita di trasformare i file di configurazione in template programmabili. Template::Toolkit rimane lo strumento per generare i documenti Compose, mentre i due `runtime.yaml` restano dati dichiarativi.

6. Precisazione sullo schema proposto

La struttura mostrata finora è una proposta di schema, non ancora una specifica completa pronta da implementare.

Prima del codice dobbiamo formalizzare:

- campi obbligatori e opzionali;
- tipi e valori ammessi;
- default;
- riferimenti disponibili;

- regole di composizione e protezione;
- preparazione e lifecycle dei volumi;
- trattamento delle chiavi sconosciute;
- comportamento per versioni dello schema non supportate.

Quindi: sì, i due file usano uno schema comune specifico di `docker-dev`; in Perl useremo `YAML::XS` per leggerli e implementeremo la semantica del formato — validazione, risoluzione e composizione — senza scrivere un parser YAML da zero.